

Chapter 15 - Dynamic Programming

1) Dynamic programming is similar to the divide-and-conquer method in that it decomposes a problem into subproblems, solves the subproblems, and then combines the solutions to subproblems to form a solution to the given problem.

Divide-and-conquer algorithms partition a problem into independent subproblems, solve the subproblems recursively, and then combine their solutions to solve the original problem.

In contrast, dynamic programming is applicable when the subproblems are not independent, that is, when subproblems share "sub sub problems". In this context a divide-and-conquer algorithm would do more work than necessary, repeatedly solving the common subproblems.

A dynamic programming algorithm solves every subproblem just once and then saves its answer in a table, thereby avoiding the work of recomputing the answer every time the subproblem is encountered.

Value of an optimal solution in terms of values of optimal solutions to the subproblems plus the cost of making the choice. ①

2) Dynamic-programming is typically applied to optimization problems. In such problems there can be many possible solutions. Each solution has a value and we wish to find a solution with the optimal (i.e. maximum or minimum) value. Such a solution is called "an" optimal solution to the problem rather than "the" optimal solution, since there can be several solutions that achieve the optimal value.

3) The development of a dynamic-programming algorithm can be broken down into a sequence of 4 steps:-

- * (i) Characterize the structure of an optimal solution.
- (ii) Recursively define the value of an optimal solution.
- (iii) Compute the value of an optimal solution in a bottom-up fashion.
- (iv) Construct an optimal solution from computed information.

Step 4 can be omitted if only the value of an optimal solution is required.

* It is better to combine steps 1 & 2, i.e. define an optimal solution to the given problem in terms of optimal solutions to subproblems and a choice. Then, recursively define the K^{th}

4) Assembly-line scheduling:

→ Some factory produces automobiles using 2 assembly lines. An automobile chassis enters each assembly line, has parts added to it at a number of stations, and a finished auto exits at the end of the line.

Each assembly has n stations numbered $j = 1, 2, \dots, n$. The j th station on i th line is denoted by $S_{i,j}$ (where $i = 1$ or 2). j th station on line 1, i.e. $S_{1,j}$, performs the same function as the j th station on line 2, i.e. $S_{2,j}$. However, the times taken by $S_{1,j}$ and $S_{2,j}$ may be different.

The assembly time required at station $S_{i,j}$ is $a_{i,j}$. The time taken by a chassis to enter assembly line i is e_i and the time a completed auto takes to exit assembly line i is x_i .

To complete the assembly in fastest possible time the factory manager may switch a partially-completed auto from one assembly line to the other. The time to transfer a chassis away from assembly line i after having gone through station $S_{i,j}$ is denoted by $t_{i,j}$ ($i = 1, 2$ and $j = 1, 2, \dots, n-1$, j cannot be equal to n since after the n th station assembly

is complete).

The problem is to determine which stations to choose from line 1 and which to choose from line 2 in order to minimize the total time through the factory for one auto.

The simplest way of determining the fastest way through the factory is to enumerate all possible ways and compute how long each way takes. But this "brute force" way of minimizing the time through the factory is infeasible when there are many stations.

If there are n stations, there are 2^n possible ways to choose stations. Thus, the brute force method would require $\Omega(2^n)$ time, which is infeasible when n is large.

Step 1: Characterize the structure of an optimal solution

Let us consider the fastest possible way for a chassis to get from the starting point through station $S_{1,j}$.

If $j=1$ there is only one possible way. But for $j=2, 3, \dots, n$ there are 2 choices:-

The fastest way through station $S_{1,j}$ is either:-

- (i) the fastest way through station $S_{1,j-1}$ and then directly through station $S_{1,j}$, or
- (ii) the fastest way through station $S_{2,j-1}$, a transfer from line 2 to line 1, and then through station $S_{1,j}$.

If the fastest way through station $S_{1,j}$ is through station $S_{1,j-1}$ then the chasis must have taken a fastest way from the starting point through station $S_{1,j-1}$. The reason being that if there were a faster way through station $S_{1,j-1}$, we could substitute this faster way to get a faster way through $S_{1,j}$ as compared to the one currently available: a contradiction.

Similarly if the fastest way through station $S_{2,j}$ is through $S_{2,j-1}$ then the chasis must have taken ^{the} fastest way from starting point through station $S_{2,j-1}$.

→ Property of optimal substructure:

An optimal solution to a problem contains within it an optimal solution to subproblems. (For example: finding the fastest way through station $S_{1,j}$ contains within it the fastest way through either $S_{1,j-1}$ or $S_{2,j-1}$).

(5)

Step 2: define the value of an optimal solution recursively in terms of optimal solutions to subproblems

Let $f_i[j]$ denote the fastest possible time to get a chassis from the starting point through S_{ij} .
 Let f^* denote the fastest time to get a chassis all the way through the factory.

Then we have:-

$$f_1[j] = \begin{cases} e_1 + a_{1,1} & \text{if } j=1 \\ \min(f_1[j-1] + a_{1,j}, f_2[j-1] + t_{2,j-1} + a_{1,j}) & \text{if } j \geq 2 \end{cases}$$

$$f_2[j] = \begin{cases} e_2 + a_{2,1} & \text{if } j=1 \\ \min(f_2[j-1] + a_{2,j}, f_1[j-1] + t_{1,j-1} + a_{2,j}) & \text{if } j \geq 2 \end{cases}$$

and,

$$f^* = \min(f_1[n] + x_1, f_2[n] + x_2)$$

The $f_i[j]$ values give the values of optimal solutions to subproblems. To help us keep track of how to construct an optimal solution, let us define $l_i[j]$ to be the line number 1 or 2 whose station $j-1$ is used in a fastest way through S_{ij} . Here $j = 2, 3, \dots, n$. $l_i[1]$ does not exist because no station precedes station 1.

We define l^* to be the line whose n th station is used in a fastest way through the entire factory.

Step 3: Compute the value of an optimal solution in a bottom-up fashion

Using the above 2 recursences we can simply write a recursive algorithm to compute the fastest way through the factory. But such a recursive algorithm would be exponential in n .

If $r_i(j)$ denotes the no. of references made to $f_i[j]$ in a recursive algorithm, we have:-

$$r_1(j) = r_2(j) = \begin{cases} 1, & \text{for } j = n \text{ (last station is called only by } f^*) \\ r_1(j+1) + r_2(j+1), & \text{otherwise} \end{cases}$$

Each value of $f_i[j+1]$ depends only on values of $f_1[j]$ and $f_2[j]$. Thus, either of $f_1[j]$ and $f_2[j]$ is called by both $f_1[j+1]$ and $f_2[j+1]$.

$$\text{Hence, } r_1(j) + r_2(j) = r_1(j+1) + r_2(j+1)$$

~~Thus~~ Thus, $r_i(j) = 2^{n-j}$. Hence, the total no. of references to all $f_i[j]$ values is $\Theta(2^n)$.

We can compute the fastest way through the factory in $\Theta(n)$ time if we compute the $f_i[j]$ values in order of increasing station

numbers and store the values in a table for later use.

FASTEST-WAY(a, t, c, x, n)

```
1   $f_1[1] \leftarrow c_1 + a_{1,1}$ 
2   $f_2[1] \leftarrow c_2 + a_{2,1}$ 
3  for  $j \leftarrow 2$  to  $n$ 
4      do if  $f_1[j-1] + a_{1,j} \leq f_2[j-1] + t_{2,j-1} + a_{1,j}$ 
5          then  $f_1[j] \leftarrow f_1[j-1] + a_{1,j}$ 
6               $l_1[j] \leftarrow 1$ 
7          else  $f_1[j] \leftarrow f_2[j-1] + t_{2,j-1} + a_{1,j}$ 
8               $l_1[j] \leftarrow 2$ 
9          if  $f_2[j-1] + a_{2,j} \leq f_1[j-1] + t_{1,j-1} + a_{2,j}$ 
10             then  $f_2[j] \leftarrow f_2[j-1] + a_{2,j}$ 
11                  $l_2[j] \leftarrow 2$ 
12             else  $f_2[j] \leftarrow f_1[j-1] + t_{1,j-1} + a_{2,j}$ 
13                  $l_2[j] \leftarrow 1$ 
14  if  $f_1[n] + x_1 \leq f_2[n] + x_2$ 
15     then  $f^* \leftarrow f_1[n] + x_1$ 
16          $l^* \leftarrow 1$ 
17     else  $f^* \leftarrow f_2[n] + x_2$ 
18          $l^* \leftarrow 2$ 
```


step 4: Construct an optimal solution from computed information

The following algorithm prints out the stations used, in decreasing order of station number

PRINT-STATIONS (l, n)

```
1   $i \leftarrow l^*$ 
2  print "line"  $i$  "station"  $n$ 
3  for  $j \leftarrow n$  downto 2
4      do  $i \leftarrow li[j]$ 
5          print "line"  $i$  "station"  $j-1$ 
```

5) Matrix-chain multiplication

→ Given a chain (or sequence) $\langle A_1, A_2, \dots, A_n \rangle$ of n matrices to be multiplied, we wish to compute the product $A_1 A_2 A_3 \dots A_n$.

→ Fully parenthesized matrix product:

A product of matrices is said to be fully parenthesized if it is either a single matrix or the product of 2 fully parenthesized matrix products, surrounded by parentheses.

→ Matrix multiplication is associative, so all parenthesizations yield the same product. 9

For example if the chain of matrices is $\langle A_1, A_2, A_3 \rangle$
we have the following parenthesizations:-

$$((A_1 A_2) A_3) \text{ and } (A_1 (A_2 A_3))$$

→ The standard algorithm for multiplication of 2 matrices is given below:-

MATRIX-MULTIPLY(A, B)

```
1  if columns[A]  $\neq$  rows[B]
2      then error "incompatible dimensions"
3  else for i  $\leftarrow$  1 to rows[A]
4      do for j  $\leftarrow$  1 to columns[B]
5          do C[i,j]  $\leftarrow$  0
6              for k  $\leftarrow$  1 to columns[A]
7                  do C[i,j]  $\leftarrow$  C[i,j] +
                              A[i,k]  $\cdot$  B[k,j]
8  return C
```

→ If A is a $p \times q$ matrix and B is a $q \times r$ matrix then C is a $p \times r$ matrix. The time taken to compute C is dominated by the no. of scalar multiplications in line 7, which is pqr .

→ Different parenthesizations of a matrix product incur different costs. Consider a chain $\langle A_1, A_2, A_3 \rangle$ of 3 matrices of dimensions:
 $10 \times 100, 100 \times 5, 5 \times 50$.

i) multiplication according to parenthesization

$((A_1 A_2) A_3)$:

dimension of $A_1 A_2 = 10 \times 5$

no. of scalar multiplications in computing

$$A_1 A_2 = 10 \times 100 \times 5 = 5000$$

no. of scalar multiplications in multiplying

$$A_1 A_2 \text{ with } A_3 = 10 \times 5 \times 50 = 2500$$

Total no. of scalar multiplications = 7500

ii) multiplication according to parenthesization

$(A_1 (A_2 A_3))$:

dimension of $A_2 A_3 = 100 \times 50$

no. of scalar multiplications in computing

$$A_2 A_3 = 100 \times 5 \times 50 = 25,000$$

no. of scalar multiplications in multiplying

$$A_1 \text{ with } A_2 A_3 = 10 \times 100 \times 50 = 50,000$$

Total no. of scalar multiplications = 75,000

Clearly, multiplication according to the first parenthesization is 10 times faster.

→ **Matrix-chain multiplication problem:**

Given a chain $\langle A_1, A_2, \dots, A_n \rangle$ of n matrices, fully parenthesize the product $A_1 A_2 A_3 \dots A_n$ in such a way that minimizes

the no. of scalar multiplications.

We suppose any matrix A_i (for $i \in [1, n]$) has dimension $P_{i-1} \times P_i$.

→ In the matrix-chain multiplication problem, we do not actually multiply the matrices.

We only determine an order for multiplying the matrices which has the lowest cost.

The time invested in determining this optimal order is more than compensated by ^{the} time saved when actually performing the matrix multiplications.

→ **Brute-force method:** The brute-force method comprises of exhaustively checking all the possible parenthesizations.

Let $P(n)$ denote the number of parenthesizations of a chain of n matrices. When $n=1$ there is just one way to fully parenthesize a matrix product. When $n \geq 2$, a fully parenthesized matrix product is the product of 2 fully parenthesized matrix subproducts. The split between the 2 subproducts may occur between the k th and $(k+1)$ th matrices where $k \in [1, n-1]$. Thus,

$$P(n) = \begin{cases} 1, & n=1 \\ \sum_{k=1}^{n-1} P(k) P(n-k), & n \geq 2 \end{cases}$$

The solution to this recurrence is $\Omega(2^n)$ i.e. exponential in n . Therefore, the brute force method is a poor strategy.

Step 1: Characterize the structure of an optimal parenthesization

Let $A_{i...j}$ denote the matrix which results from evaluating the product $A_i A_{i+1} A_{i+2} \dots A_j$.

Any parenthesization of $A_i \dots A_j$ must split the product between A_k and A_{k+1} for some integer k in the range $i \leq k < j$. We first compute the optimal parenthesizations of $A_i A_{i+1} \dots A_k$ and $A_{k+1} A_{k+2} \dots A_j$ and use them to compute the optimal parenthesization of $A_i A_{i+1} \dots A_j$.

The optimal substructure of this problem is as follows:-

Suppose that an optimal parenthesization of $A_i A_{i+1} \dots A_j$ splits the product between A_k and A_{k+1} . Then the parenthesization of the "prefix subchain" $A_i A_{i+1} \dots A_k$ within this optimal parenthesization must itself be an optimal parenthesization. Similarly, the parenthesization of the "postfix subchain" $A_{k+1} A_{k+2} \dots A_j$ must also be an optimal parenthesization.

Step 2: Recursively define the value of an optimal solution

Let $m[i, j]$ denote the minimum no. of scalar multiplications needed to compute the matrix $A_{i..j}$.

If $i = j$, $A_{i..i} = A_i$. No scalar multiplication is required. Thus $m[i, i] = 0$ for $i \in [1, n]$.

$m[i, j]$ is equal to the minimum cost for computing the subproducts $A_{i..k}$ and $A_{k+1..j}$, plus the cost for multiplying these two matrices together.

Note that for the chain $\langle A_1, A_2, A_3, \dots, A_n \rangle$ to be compatible for multiplication:

no. of columns in $A_1 =$ no. of rows in A_2

columns $[A_2] =$ rows $[A_3]$

$\vdots$

columns $[A_{n-1}] =$ rows $[A_n]$

We define the dimension of A_i to be $p_{i-1} \times p_i$.

Thus:-

$$m[i, j] = m[i, k] + m[k+1, j] + p_{i-1} \times p_k \times p_j$$

Where, k could take the values

$$k = i, i+1, i+2, \dots, j-1$$

Thus, for an optimal solution:-

$$m[i, j] = \begin{cases} 0 & \text{if } i = j \\ \min_{i \leq k < j} \{ m[i, k] + m[k+1, j] + p_{i-1} p_k p_j \} & \text{if } i < j \end{cases}$$

We define $s[i, j]$ to be that value of k at which we split the product $A_i A_{i+1} \dots A_j$ to obtain an optimal parenthesization.

Step 3: Compute the value of an optimal solution in a bottom-up fashion

If the dimension of a matrix A_i is

$p_{i-1} \times p_i$ the procedure MATRIX-CHAIN-ORDER takes as input the array p of length $n+1$.

$$p = \langle p_0, p_1, p_2, p_3, \dots, p_n \rangle$$

MATRIX-CHAIN-ORDER(p)

```
1   $n \leftarrow \text{length}[p] - 1$ 
2  for  $i \leftarrow 1$  to  $n$ 
3      do  $m[i, i] \leftarrow 0$ 
4  for  $\text{length} \leftarrow 2$  to  $n$   $\triangleright$   $\text{length}$  is the chain length
5      do for  $i \leftarrow 1$  to  $n - \text{length} + 1$ 
6          do  $j \leftarrow i + \text{length} - 1$ 
7               $m[i, j] \leftarrow \infty$ 
8                  for  $k \leftarrow i$  to  $j - 1$ 
9                      do  $\text{cost} \leftarrow m[i, k] + m[k + 1, j]$ 
                           $+ p_{i-1} * p_k * p_j$  15
```

10 if $\text{cost} < m[i, j]$
 11 then $m[i, j] \leftarrow \text{cost}$
 12 $s[i, j] \leftarrow k$
 13 return m and s

→ The procedure MATRIX-CHAIN-MULTIPLICATION computes the $m[i, j]$ values for chains of increasing length.

First of all, it sets the costs for chains of length 1 to 0 by executing $m[i, i] \leftarrow 0$ for all $i \in [1, n]$.

Next, it computes the costs for chains of length 2 i.e. it computes $m[i, j]$ where $j = i+1$ (i.e. $m[i, i+1]$) for all i varying

from 1 to $n-1$.

Note that when you are computing the cost for a matrix of length "length" the 2nd index in $m[i, j]$ i.e. j should be such that

$$j - i + 1 = \text{length}, \text{ or } j = i + \text{length} - 1.$$

Secondly, the value of i cannot approach the upper boundary of n with a distance shorter than "length". Thus i varies from 1 to $n - \text{length} + 1$. (otherwise $j = i + \text{length} - 1$ would exceed the upper bound of n).

→ MATRIX-CHAIN-ORDER has a running time of $O(n^3)$ which is much more efficient than the brute-force exponential-time method.

Step 4: Constructing an optimal solution

PRINT-OPTIMAL-PARENTHEZIZATION(s, i, j)

```
1  if  $i = j$ 
2      then print "A"  $i$ 
3  else print "("
4      PRINT-OPTIMAL-PARENTHEZIZATION
        ( $s, i, s[i, j]$ )
5      PRINT-OPTIMAL-PARENTHEZIZATION
        ( $s, s[i, j] + 1, j$ )
6  print ")"
```

Continuation from 26 pages before

6) Elements of dynamic Programming

Any optimization problem must have 2 key ingredients ^{in order} for dynamic programming to be applicable: optimal substructure and overlapping subproblems.

1) Optimal substructure: A problem is said to exhibit optimal substructure if an optimal solution to the problem contains within it optimal solutions to subproblems.

When a problem exhibits optimal substructure both dynamic programming and greedy strategies may apply.

A common pattern exists in discovering optimal substructure:-

(i) You show that a solution to the problem consists of making a choice, such as choosing a preceding assembly-line station or choosing an index at which to split the matrix chain.

Making this choice leaves you with one or more subproblems to be solved.

(ii) You next suppose that for a given subproblem you are given the choice that leads to an optimal solution. You are not concerned with how to determine this choice. You just assume that it has been given to you.

(iii) Given this choice, you determine which subproblems ensue and how to best characterize the resulting space of subproblems.

(iv) You show that the solutions to the subproblems used within the optimal solution to the problem must themselves be optimal by using a "cut-and-paste" technique. By "cutting out" the non-optimal subproblem solution and "pasting in" the optimal one, you show that you can get a better solution to the original problem, thus contradicting your supposition that you already had an optimal solution.

Dynamic Programming uses optimal substructure in a bottom up fashion. We first find optimal solutions to the subproblems, and having solved the subproblems, we find an optimal solution to the problem.

Finding an optimal solution to a problem entails making a choice among the subproblems which we will use in solving the problem. The cost of the problem solution is usually the subproblem costs plus a cost that is directly attributable to the choice itself. For example, in matrix-chain multiplication we first determined the optimal parenthesizations of subchains of various lengths of $A_1 A_2 \dots A_j$ and then we chose the matrix A_k at which to split the product. The cost attributable to the choice itself is the term $p_{i-1} p_k p_j$.

Substutitlins in optional Substructure:

One should be careful not to assume that optional substructure applies when it actually does not. Consider the following 2 directed graph problems:-

- (i) Unweighted shortest path: Find the shortest path from vertex u to vertex v . Such a path must be simple since removing a cycle from a path produces a path with fewer edges.
- (ii) Unweighted longest simple path: Find the longest simple path from u to v . If the path is not simple we can traverse a cycle as many times as we like to create paths of arbitrarily large no. of edges.

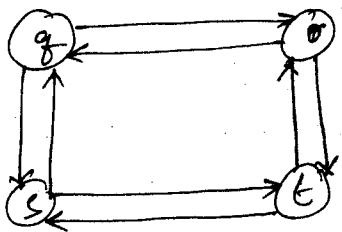
The unweighted shortest path problem exhibits optional substructure: any shortest path p from u to v must contain an intermediate vertex w (w may be u or v). Thus the path $u \xrightarrow{p} v$ comprises of the subpaths $u \xrightarrow{p_1} w \xrightarrow{p_2} v$. No. of edges in $p = \text{No. of edges in } p_1 + \text{No. of edges in } p_2$. If p is a shortest path from u to v , then p_1 must be a shortest path from u to w (otherwise we could "cut out" p_1 and "paste" the shortest path p_1' from u to w , thus contradicting p 's optimality). Similarly, p_2 must be an optimal path from w to v .

It is tempting to assume that the problem of finding an unweighted longest simple path exhibits optional substructure as well.

If we decompose a longest simple path $u \xrightarrow{p} v$ into subpaths $u \xrightarrow{p_1} w \xrightarrow{p_2} v$, then p_1 and p_2

may not be the longest simple paths from u to w and w to v respectively. The reason being that if we find the longest simple paths from u to w and w to v and then join them, the resulting path from u to v may no longer be simple.

eg:-



The path $q \rightarrow r \rightarrow t$ is a longest simple path from q to t . But the subpath $q \rightarrow r$ is not a longest simple path from q to r , nor is the subpath $r \rightarrow t$ a longest simple path from r to t .

If we combine the longest simple paths :-

(a) $q \rightarrow s \rightarrow t \rightarrow r$ (q to r), and

(b) $r \rightarrow q \rightarrow s \rightarrow t$ (r to t).

The resulting path $q \rightarrow s \rightarrow t \rightarrow r \rightarrow q \rightarrow s \rightarrow t$ is no longer simple.

This happens because the subproblems in finding the longest simple path are not independent. Two subproblems "of the same problem" are said to be independent if the solution to one subproblem does not affect the solution to another subproblem. In the present case our use of some resources in solving one subproblem has rendered them unavailable for the other subproblem [the

longest path $q \rightarrow s \rightarrow t \rightarrow r$ from q to r uses vertices s and t . Hence they cannot be used in the longest path from r to t otherwise the resulting path would not be simple.]

But the subproblems for the shortest-path problem are independent: Any shortest path $u \xrightarrow{p} v$ ~~may contain~~ at can be partitioned at w as $u \xrightarrow{p_1} w \xrightarrow{p_2} v$ (w may be $= u$ or v).

The shortest paths p_1 and p_2 may not contain any common vertex except the vertex of partitioning (w is certainly allowed). Suppose some vertex $x \neq w$ appears in both p_1 and p_2 , so that p_1 is $u \xrightarrow{p_{u,x}} x \xrightarrow{p_{x,w}} w$ and p_2 is $w \xrightarrow{p_{w,x}} x \xrightarrow{p_{x,v}} v$. Then the path

$u \xrightarrow{p_{u,x}} x \xrightarrow{p_{x,v}} v$ from u to v would be shorter than p by at least 2 edges (thus contradicting p 's optimality).

Thus, we are assured that the subproblems for shortest path problem are optimal.

2) Overlapping subproblems:

The second ingredient an optimization problem must have for dynamic programming to be applicable is that the space of subproblems must be small in the sense that a recursive algorithm for the problem solves the same subproblems over and over, rather than always generating new subproblems. Typically, the total no. of distinct subproblems is a polynomial in input size. An optimization problem is said

to have overlapping subproblems if a recursive solution to the problem revisits the same subproblems over and over again.

In contrast, a problem for which divide-and-conquer approach is suitable generates brand new subproblems at each step of the recursion.

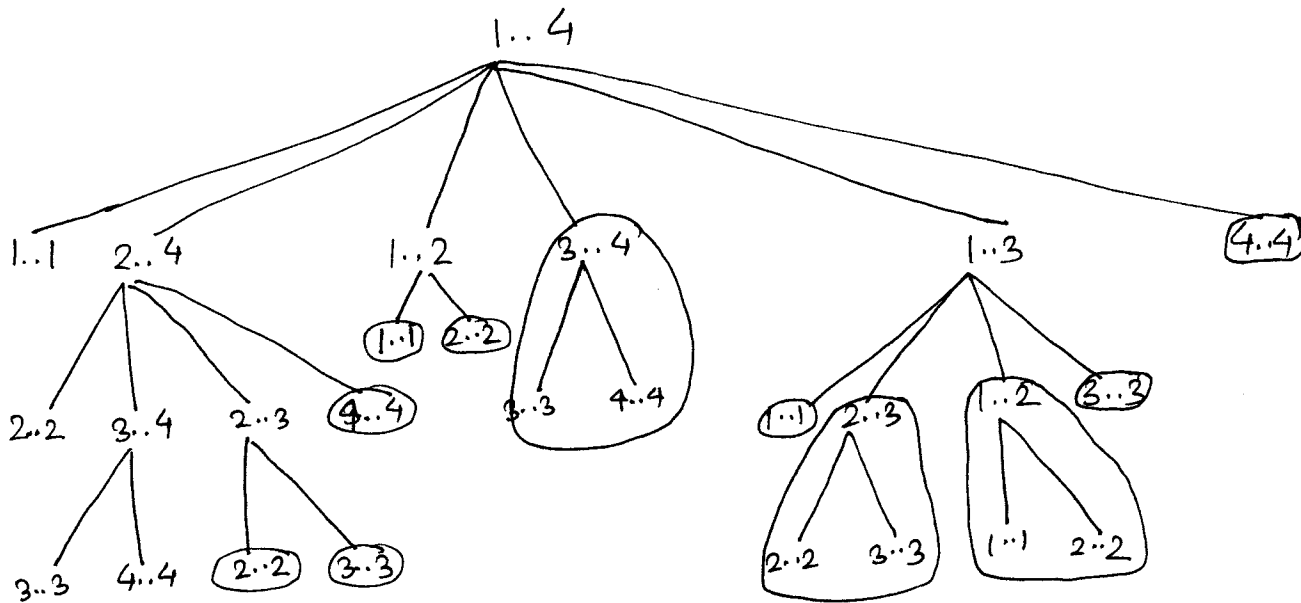
Dynamic programming takes advantage of overlapping subproblems by solving each subproblem once and then storing the solutions in a table where it can be looked up when needed.

Consider the following recursive procedure for computation of $m[1..n, 1..n]$:-

RECURSIVE-MATRIX-CHAIN (p, i, j) $\triangleright$ computes only $m[i, j]$

- 1 if $i = j$
then return 0
- 2 $m[i, j] \leftarrow \infty$
- 3 for $k \leftarrow i$ to $j-1$
- 4 do $cost \leftarrow$ RECURSIVE-MATRIX-CHAIN(p, i, k)
+ RECURSIVE-MATRIX-CHAIN($p, k+1, j$)
+ $p_{i-1} p_k p_j$
- 5 if $cost < m[i, j]$
- 6 then $m[i, j] \leftarrow cost$
- 7 return $m[i, j]$

The recursion tree produced by the call
 $\text{RECURSIVE-MATRIX-CHAIN}(p, 1, 4)$ is as follows: —



Clearly, for the solution of a single problem involves revisiting the same subproblems over and over again. The circled subproblems represent those subproblems that the simple recursion based algorithm solves again.

The recursion of this algorithm is exponential in n . This recursive algorithm uses a top-down approach. It repeatedly solves each subproblem each time it reappears in the recursion tree.

8) Longest Common Subsequence:

→ A subsequence of a given sequence is just the given sequence with zero or more elements left out. Thus if $X = \langle x_1, x_2, x_3, \dots, x_m \rangle$ is a sequence, then $\langle x_1, x_2, x_5 \rangle$, $\langle x_3, x_m \rangle$ are 2 subsequences of X .

→ Given two sequences X and Y , we say that a sequence Z is a common subsequence of X and Y if Z is a subsequence of both X and Y .

→ longest-common-subsequence problem:

Given 2 sequences $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$ we wish to find a maximum-length common subsequence of X and Y .

step 1: characterize the structure of an optimal solution to the problem

→ A simple brute force method is to enumerate all possible subsequences of X and check each subsequence to check if it is also a subsequence of Y . Since ~~the~~ ^{each} subsequence of X corresponds to a subset of the indices $\{1, 2, 3, \dots, m\}$ of X there are 2^m possible subsequences of X . Hence, this approach requires exponential time.

→ Given a sequence $X = \langle x_1, x_2, x_3, \dots, x_m \rangle$ we define the i th prefix of X as $X_i = \langle x_1, x_2, \dots, x_i \rangle$ where $i = 0, 1, 2, \dots, m$. X_0 is the empty sequence.

→ Theorem (optimal substructure of an LCS)

Let $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$ be 2 sequences and let $Z = \langle z_1, z_2, \dots, z_k \rangle$ be any LCS of X and Y .

1) If $x_m = y_n$, then $z_k = x_m = y_n$ and z_{k-1} is an LCS of X_{m-1} and Y_{n-1} .

Proof: If $x_m = y_n$ but $z_k \neq x_m$, then we could append $x_m = y_n$ to Z to obtain a common subsequence of X and Y of length $k+1$, thus contradicting the supposition that Z is a longest common subsequence of X and Y . Hence, $z_k = x_m = y_n$.

The prefix z_{k-1} is a ^{length-} $(k-1)$ common subsequence of X_{m-1} and Y_{n-1} . z_{k-1} must be an LCS. Suppose this is not true and W is a common subsequence of X_{m-1} and Y_{n-1} that has length $> k-1$. Then appending $x_m = y_n$ to W produces a common subsequence of X and Y of length greater than k , which is a contradiction.

2) If $x_m \neq y_n$, then $z_k \neq x_m$ implies that Z is an LCS of X_{m-1} and Y .

Proof: If $z_k \neq x_m$ then Z is a common subsequence of X_{m-1} and Y_n . If there were a common subsequence of X_{m-1} and Y_n (say W) with length $> k$ then it must also have been a common subsequence of X_m and Y_n contradicting the supposition that Z is an LCS of X and Y .

3) If $x_n \neq y_n$, then $z_k \neq y_n$ implies that z_k is an LCS of X and Y_{n-1} .

Proof: Similar to above.

Step 2: recursively define the value of an optimal solution

Let $c[i, j]$ be equal to the length of an LCS of the sequences X_i and Y_j .

$$c[i, j] = \begin{cases} 0 & , \text{if either } i \text{ or } j = 0 \\ c[i-1, j-1] + 1 & , \text{if } i, j > 0 \text{ and } x_i = y_j \\ \max(c[i, j-1], c[i-1, j]) & , \text{if } i, j > 0 \\ & \text{and } x_i \neq y_j \end{cases}$$

Note that a condition in the problem restricts which subproblems we may consider. When $x_i = y_j$ we consider the problem of finding the LCS of X_{i-1} and Y_{j-1} . Otherwise, we consider the 2 subproblems of finding the LCS of X_i and Y_{j-1} and of X_{i-1} and Y_j .

Step 3: Compute the length of an LCS in a bottom up fashion

In an attempt to find an LCS of X_i and Y_j there are m different X_i 's and n different Y_j 's. Thus the total no. of subproblems = $\Theta(mn)$.

The procedure `LCS-LENGTH` stores values in the table $c[0..m, 0..n]$ in a row-major order. (That is, the 1st row of c is filled in from left to right, then the second row and so on).

It also initializes a table $b[1..m, 1..n]$ to simplify construction of an optimal solution. At the end $c[m, n]$ holds the value of the length of an LCS of X and Y .

LCS-LENGTH(X, Y)

```
1   $m \leftarrow \text{length}[X]$ 
2   $n \leftarrow \text{length}[Y]$ 
3  for  $i \leftarrow 1$  to  $m$ 
4      do  $c[i, 0] \leftarrow 0$ 
5  for  $j \leftarrow 1$  to  $n$ 
6      do  $c[0, j] \leftarrow 0$ 
7  for  $i \leftarrow 1$  to  $m$ 
8      do for  $j \leftarrow 1$  to  $n$ 
9          do if  $x_i = y_j$ 
10             then  $c[i, j] \leftarrow c[i-1, j-1] + 1$ 
11                  $b[i, j] \leftarrow \text{"\u2191"}$ 
12             else if  $c[i-1, j] \geq c[i, j-1]$ 
13                 then  $c[i, j] \leftarrow c[i-1, j]$ 
14                      $b[i, j] \leftarrow \text{"\u2191"}$ 
15             else  $c[i, j] \leftarrow c[i, j-1]$ 
16                  $b[i, j] \leftarrow \text{"\u2190"}$ 
17  return  $c$  and  $b$ 
```

Note that the value of $c[i,j]$ depends only upon the previously computed values of $c[i-1,j]$, $c[i,j-1]$, $c[i-1,j-1]$.

Whenever $x_i = y_j$, that character is an element of an LCS. Hence, $b[i,j]$ contains a " $\uparrow$ ". Otherwise $b[i,j]$ contains either a " $\uparrow$ " or a " $\leftarrow$ ". Starting from (m,n) index of the table, these arrows help to find the characters that are part of an optimal solution.

eg:- $X = \langle A, B, C, B, D, A, B \rangle$ and $Y = \langle B, D, C, A, B, A \rangle$

		0	1	2	3	4	5	6
	y_j	B	D	C	A	B	A	
0	x_i	0	0	0	0	0	0	
1	A	0	$\uparrow$	$\uparrow$	$\uparrow$	$\nwarrow$	$\nwarrow$	$\nwarrow$
2	B	0	$\nwarrow$	$\nwarrow$	$\nwarrow$			
3	C	0			$\uparrow$	$\nwarrow$	$\nwarrow$	
4	B	0				$\uparrow$		
5	D	0				$\uparrow$		
6	A	0					$\nwarrow$	
7	B	0					$\uparrow$	

All entries have not been shown.

→ Running time of the procedure is $O(mn)$.

Step 4: Constructing an LCS

PRINT-LCS (b, X, i, j) $\triangleright$ initial call (b, X, length[X], length[Y])

```
1  if  $i = 0$  or  $j = 0$ 
2      then return
3  if  $b[i, j] = "\uparrow"$ 
4      then PRINT-LCS (b, X,  $i-1, j-1$ )
5          print  $x_i$            $\triangleright x_i = X[i]$ 
6  elseif  $b[i, j] = "\uparrow"$ 
7      then PRINT-LCS (b, X,  $i-1, j$ )
8  else PRINT-LCS (b, X,  $i, j-1$ )
```

→ At each stage of the recursion at least one of i and j is decremented as you move in the b table from bottom-right corner to top-left corner. Hence the runtime of the procedure is $O(m+n)$.

9) Optimal binary search trees:

→ Suppose a program to translate text from English to French. For each occurrence of each English word in the text, we need to lookup its French equivalent. One way to perform these lookup operations is to build a binary search tree with English words as keys and their French equivalents as satellite data.

We want the total time spent searching to be as low as possible. Any balanced binary search tree could ensure an $O(\lg n)$ search time per occurrence. (such as red-black tree)

However, words appear with different frequencies and it may be the case that a frequently used word appears far from the root, while a rarely used word appears near the root. Since the no. of nodes visited when searching for a key in a binary search tree is one plus the depth of the key, such an organization would slow down translation. Frequently occurring words should be placed near the root. Moreover, there may be such words for which there is no French translation.

→ We need to organize a binary search tree in such a fashion that minimizes the total no. of nodes visited in all searches, given that we know how often each word occurs. What we need is known as an optimal binary search tree.

→ Given a sequence $K = \langle K_1, K_2, K_3, \dots, K_n \rangle$ of n distinct keys in sorted order. Associated with every key K_i is a probability P_i that a search will be for K_i . For searches for values not in K we have $n+1$ dummy keys $d_0, d_1, d_2, \dots, d_n$. d_0 represents all values less than K_1 , d_n represents all values greater than K_n , every other d_i

represent all values between K_i and K_{i+1} .
The deemonies have probabilities q_i 's associated with them.

If a search results in finding some key K_i , it's a successful search. If a search results in finding some deemon key d_i , it's an unsuccessful search.

→ We have: -

$$\sum_{i=1}^n p_i + \sum_{i=0}^n q_i = 1$$

If we define the actual cost of a search to be the no. of nodes examined, the expected cost of a search in a binary search tree would then be: -

$$\begin{aligned} E[\text{search cost in } T] &= \sum_{i=1}^n (\text{depth}(K_i) + 1) \cdot p_i + \sum_{i=0}^n (\text{depth}(d_i) + 1) \cdot q_i \\ &= 1 + \sum_{i=1}^n \text{depth}(K_i) \cdot p_i + \sum_{i=0}^n \text{depth}(d_i) \cdot q_i \end{aligned}$$

→ An optimal binary search tree is a binary search tree with minimum expected search cost for a given set of probabilities.

An optimal binary search tree need not necessarily have the smallest height for the given set of keys. Nor does an optimal binary search

tree necessarily contain the key with the greatest probability at the root.

An exhaustive checking of all possibilities fails to yield an efficient algorithm since the no. of possible binary trees with n nodes is $\Omega(4^n/n^{3/2})$ and so there are an exponential no. of binary search trees.

Step 1: The structure of an optimal binary search tree

If an optimal binary search tree T has a subtree T' containing the keys $K_i \dots K_j$, then this subtree T' must be optimal as well for the subproblem with keys $K_i \dots K_j$ and dummy keys $d_{i-1} \dots d_j$. The usual cut-and-paste argument applies: If there were a subtree T'' whose expected cost is lower than that of T' , then we could cut T' out of T and paste in T'' , resulting in a binary search tree of cost lower than that of T , thus contradicting the optimality of T .

Given keys $K_i \dots K_j$, one of these keys, say K_r ($i \leq r \leq j$) will be the root of an optimal binary search tree containing these keys. The left subtree of K_r will contain the keys $K_i \dots K_{r-1}$ and dummy keys $d_{i-1} \dots d_{r-1}$ whereas the right subtree will contain the keys $K_{r+1} \dots K_j$ and dummy keys $d_r \dots d_j$.

As long as we examine all candidate roots K_r ($i \leq r \leq j$) and determine all optimal binary search trees containing $K_i \dots K_{r-1}$ and those containing $K_{r+1} \dots K_j$ we are guaranteed that we will find an optimal binary search tree.

If you let K_r be K_i then the left subtree is empty containing the keys $K_i \dots K_{i+1}$, (i.e. no valid key but a single dummy d_{i-1}).

If you let K_r be K_j then the right subtree is empty containing the keys $K_{j+1} \dots K_j$, (i.e. no valid key but a single dummy d_j).

Step 2: Recursively define the value of an optimal solution

Let $e[i, j]$ represent the expected cost of searching an optimal binary search tree containing the keys $K_i \dots K_j$. We, ultimately wish to compute $e[1, n]$.

If $j = i - 1$ we just have the dummy d_{i-1} at depth 0. Thus, $e[i, i-1] = q_{i-1}$.

What happens to the expected search cost of a subtree when it becomes a subtree of a node?

The depth of each node in the subtree 35

increases by 1. Thus, the expected search cost of this subtree increases by the sum of all the probabilities in the subtree.

Let $W(i, j)$ denote the sum of probabilities of all nodes (internal, key containing + external, dummy containing) of a subtree with keys $K_i, \dots, K_j$. Then: -

$$W(i, j) = \sum_{l=i}^j p_l + \sum_{l=i-1}^j q_l$$

For $j \geq i$ we have: -

$$\begin{aligned} e[i, j] &= p_r + \{e[i, r-1] + W(i, r-1)\} + \{e[r+1, j] + W(r+1, j)\} \\ &= e[i, r-1] + e[r+1, j] + W(i, j) \end{aligned}$$

Thus,

$$e[i, j] = \begin{cases} q_{i-1} & , \text{ if } j = i-1 \\ \min_{i \leq r \leq j} \{ e[i, r-1] + e[r+1, j] + W(i, j) \} & , \text{ if } i \leq j \end{cases}$$

For any optimal binary search tree containing the keys $K_i, \dots, K_j$ and having K_r as its root, we define $\text{root}[i, j]$ to be the index r of the root.